



UNIVERSITY OF SIALKOT

Web Engineering

Title : Semester Project Report

Submit by : **Muttahir Ali**

Roll no : 23010101- 082

Submit by : **Abdullah Waryah**

Roll no : 23010101- 093

Department : BSSE (6th Gray)

Submit to : **Prof Museb Khalid**

1. INTRODUCTION

The idea behind the app is simple: a student enters their subjects, picks a grade and credit hours for each one, and the app works out their semester GPA and overall CGPA. There's also a page showing the university's grading policy so students can check what each grade is worth before they enter anything.

2. PROJECT STRUCTURE AND ARCHITECTURE

We stuck with the default Vue 3 + Vite folder layout rather than reinventing it. Everything sits under `src/`, split into assets, components, views, and router. Keeping views (full pages tied to a route) separate from components (smaller pieces reused inside those pages) made it a lot easier to keep track of what belongs where as the project grew.

```
src/  
├─ assets/  
│   ├─ base.css, main.css, style.css  
│   └─ logo.svg  
├─ components/  
│   ├─ Navbar.vue  
│   ├─ Footer.vue  
│   ├─ SubjectRow.vue  
│   ├─ ResultCard.vue  
│   └─ PreviousCGPA.vue  
│  
│  
├─ router/  
│   └─ index.js  
├─ views/  
│   ├─ CombineView.vue  
│   └─ GradingPolicyView.vue  
│  
├─ App.vue  
└─ main.js
```

2.1 Component Hierarchy

App.vue is basically just a shell — it puts Navbar and Footer around a RouterView so they show up on every page, and RouterView swaps in whichever page is active. The calculator page (CombineView) is where most of the actual work happens: it pulls in PreviousCGPA for the student's prior record, a SubjectRow for every subject they add, and four ResultCard components to show the final numbers.

```
App
├── Navbar
├── RouterView
│   ├── CombineView ("/")
│   │   ├── PreviousCGPA
│   │   ├── SubjectRow (v-for, one per subject)
│   │   └── ResultCard (x4 – SGPA, CGPA, Sem. Hours, Total Hours)
│   └── GradingPolicyView ("/grading-policy")
└── Footer
```

3. SINGLE PAGE APPLICATION ROUTING

We're using Vue Router for navigation. It's set up in router/index.js and then hooked into the app in main.js with app.use(router). Right now there are two routes defined — one for the calculator (the home page) and one for the grading policy table. App.vue wraps everything in RouterView so switching pages doesn't reload the browser.

3.1 Router Configuration

```
// src/router/index.js
import { createRouter, createWebHistory } from 'vue-router'

import CombineView from '../views/CombineView.vue'
import GradingPolicyView from '../views/GradingPolicyView.vue'

const routes = [
  {
    path: '/',
    name: 'Combine',
    component: CombineView
  },
  {
    path: '/grading-policy',
    name: 'GradingPolicy',
```

```

      component: GradingPolicyView
    }
  ]

```

```

const router = createRouter({
  history: createWebHistory(),
  routes
})

```

```

export default router

```

3.2 Application Entry Point

```

// src/main.js
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

import './assets/style.css'

createApp(App)
  .use(router)
  .mount('#app')

```

3.3 Layout Wrapper

```

<!-- src/App.vue -->
<template>
  <Navbar />

  <div class="container">
    <RouterView />
  </div>

  <Footer />
</template>

<script setup>
import Navbar from './components/Navbar.vue'
import Footer from './components/Footer.vue'
</script>

```

Navbar.vue uses RouterLink for the actual nav links, which is what gives us the SPA behaviour clicking between the calculator and grading policy pages doesn't trigger a full reload, and the active link gets highlighted automatically.

4. MODULAR COMPONENT ARCHITECTURE

Instead of cramming everything into one giant page, we broke the UI down by what each piece actually does. Navbar and Footer only get used once, at the top level. The two route-level pages live in views/, and the smaller reusable pieces — SubjectRow, ResultCard, PreviousCGPA — live in components/ and get composed together inside CombineView. Below is a quick rundown of what each one does:

Navbar.vue / Footer.vue — the layout that stays on screen the whole time. Navbar handles the RouterLink navigation.

CombineView.vue — the main calculator page. Holds the subjects array, previousCgpa, previousHours, and all the GPA math.

PreviousCGPA.vue — a small form for the student's existing CGPA and credit hours, needed to work out the overall CGPA.

SubjectRow.vue — one row per subject: grade dropdown, credit-hours input, delete button. Gets rendered once for each subject.

ResultCard.vue — a simple reusable card that just takes a title and a value; we reuse it four times for SGPA, CGPA, semester hours, and total hours.

GradingPolicyView.vue — a static page with the grading table, doesn't need any props.

5. INTER-COMPONENT COMMUNICATION

This is the part the assignment cares about most, so we made sure it's clean: data only ever flows down through props, and children only ever talk back up through emitted events. There's no shared store — CombineView keeps all the state itself and just hands pieces of it down to whichever child needs them.

5.1 Props: Parent → Child

For every subject, CombineView passes its index, grade, and creditHours down into a SubjectRow as props. SubjectRow doesn't touch these directly — it just displays them and emits an event whenever the user changes something.

```
<!-- SubjectRow.vue (child) -->
<template>
  <div class="subject-row">
    <div class="subject">Subject {{ index + 1 }}</div>

    <select :value="grade" @change="changeGrade">
      <option value="">Select Grade</option>
      <option>A+</option><option>A</option> ...
    </select>

    <input type="number" min="1" max="10"
      :value="creditHours" @input="changeCredit" />

    <button class="delete-btn" @click="$emit('remove', index)">
      Delete
    </button>
  </div>
</template>
```

```

<script setup>
const props = defineProps({
  index: Number,
  grade: String,
  creditHours: Number
})

const emit = defineEmits([
  "update-grade", "update-credit", "remove"
])

function changeGrade(event) {
  emit("update-grade", { index: props.index, grade: event.target.value })
}
function changeCredit(event) {
  emit("update-credit", {
    index: props.index, credit: Number(event.target.value)
  })
}
</script>

```

5.2 Emits: Child → Parent

On the other side, CombineView is listening for update-grade, update-credit, and remove from every SubjectRow, and updates its subjects array whenever one fires. The same pattern happens with PreviousCGPA, just with update-cgpa and update-hours instead.

```

<!-- views/CombineView.vue (parent) -->
<template>
  <PreviousCGPA :cgpa="previousCgpa" :hours="previousHours"
    @update-cgpa="previousCgpa = $event"
    @update-hours="previousHours = $event" />

  <SubjectRow
    v-for="(subject, index) in subjects" :key="index"
    :index="index" :grade="subject.grade"
    :credit-hours="subject.creditHours"
    @update-grade="updateGrade"
    @update-credit="updateCredit"
    @remove="removeSubject" />

  <ResultCard title="Semester SGPA" :value="semesterSGPA" />
  <ResultCard title="Overall CGPA" :value="overallCGPA" />
</template>

<script setup>
const subjects = ref([])

function updateGrade(data) {
  subjects.value[data.index].grade = data.grade
}
function updateCredit(data) {
  subjects.value[data.index].creditHours = data.credit
}
function removeSubject(index) {
  subjects.value.splice(index, 1)
}

```

</script>

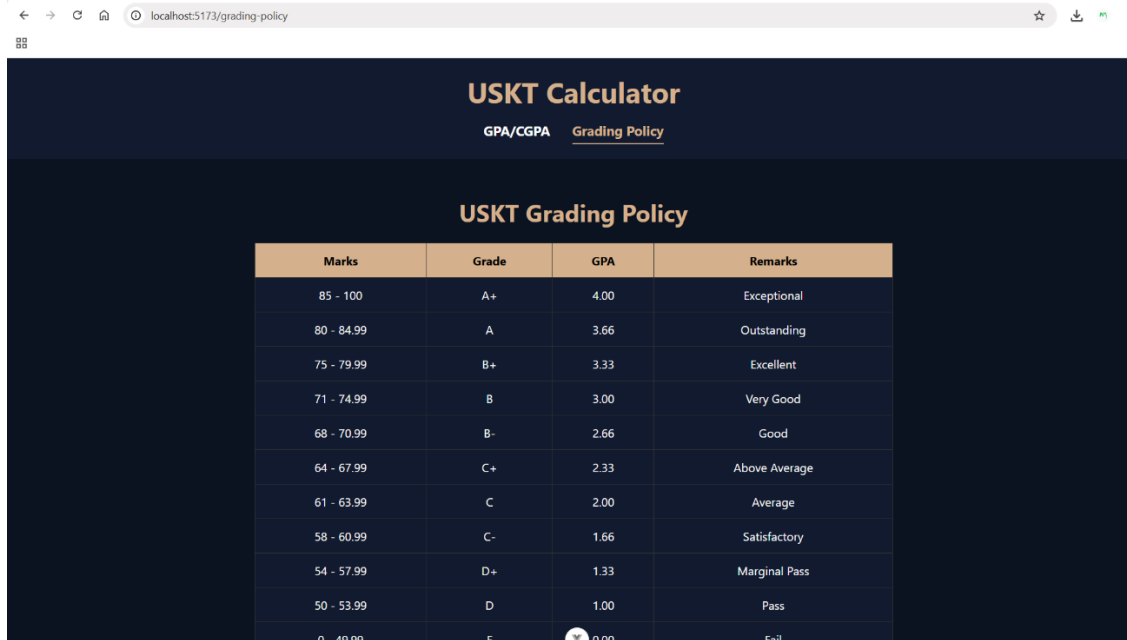
6. SCREENSHOTS OF THE RUNNING APPLICATION

To run it locally: `npm run dev`, then open whatever localhost URL shows up in the terminal We'll drop in screenshots below showing the calculator.

ScreenShot:

The screenshot shows the 'USKT Calculator' application running in a web browser at localhost:5173. The interface has a dark blue theme. At the top, there's a navigation bar with 'USKT Calculator' and two links: 'GPA/CGPA' (active) and 'Grading Policy'. Below this is a section titled 'USKT GPA / CGPA'. It contains a 'Previous Academic Record' section with two input fields: 'Previous CGPA' and 'Total Credit Hours'. Below these is a 'Number of Subjects' section with an input field and 'Generate' and 'Reset' buttons. A large orange button labeled 'Calculate GPA / CGPA' is centered below. At the bottom, there are four buttons: 'Semester SGPA', 'Overall CGPA', 'Semester Credit Hours', and 'Total Credit Hours'.

This screenshot shows the application after clicking the 'Generate' button. The 'Number of Subjects' input field now contains the value '6'. Below this, there is a table with 6 rows, each representing a subject. Each row has a 'Select Grade' dropdown menu, a 'Credit Hours' input field, and a 'Delete' button. Below the table is the same 'Calculate GPA / CGPA' orange button. At the bottom, the four buttons now display numerical values: 'Semester SGPA' is 0.00, 'Overall CGPA' is 0.00, 'Semester Credit Hours' is 0, and 'Total Credit Hours' is 0.



Marks	Grade	GPA	Remarks
85 - 100	A+	4.00	Exceptional
80 - 84.99	A	3.66	Outstanding
75 - 79.99	B+	3.33	Excellent
71 - 74.99	B	3.00	Very Good
68 - 70.99	B-	2.66	Good
64 - 67.99	C+	2.33	Above Average
61 - 63.99	C	2.00	Average
58 - 60.99	C-	1.66	Satisfactory
54 - 57.99	D+	1.33	Marginal Pass
50 - 53.99	D	1.00	Pass
0 - 49.99	F	0.00	Fail

7. CONCLUSION

That covers the client-side for Deliverable 1. Routing works across the calculator and grading policy pages, the UI is broken into reusable components instead of one big file, and props/emit keep the data flow predictable. Next step is connecting it to an Express.js and MongoDB backend so results can actually be saved.